

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

Name: T Gnaneswari

Reg no: !92372183

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1. A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach

to resolve these issues, including validation of SOAP messages and error handling techniques.
Explain how the corrected system ensures reliable communication.

Code of Conduct:

*I **T Gnaneswari(192372183)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

T Gnaneswari

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

SOAP AND WSDL WEB SERVICES

Question 1: SOAP-Based Hospital Management Web Service

A hospital management system needs secure and interoperable communication among registration, appointment, laboratory, pharmacy, billing and medical-record departments. SOAP is suitable because it uses standardized XML messages, can be formally described using WSDL, supports strong enterprise security standards, and provides structured fault handling.

Hospital SOAP Web Service Architecture



Figure 1. Hospital SOAP web service architecture.

Proposed Operations

Operation	Purpose
AddPatient	Registers a new patient and returns a unique patient ID.
GetPatientDetails	Retrieves demographic and registration information.
ScheduleAppointment	Books a patient with a doctor at a selected date and time.
UpdateAppointment	Changes appointment date, time or doctor.
CancelAppointment	Cancels an existing appointment.
GetMedicalRecords	Retrieves diagnoses, treatments, prescriptions and visit history.

UpdateMedicalRecord	Adds or updates authorized clinical information.
---------------------	--

SOAP Message Structure

Structure of a SOAP Message

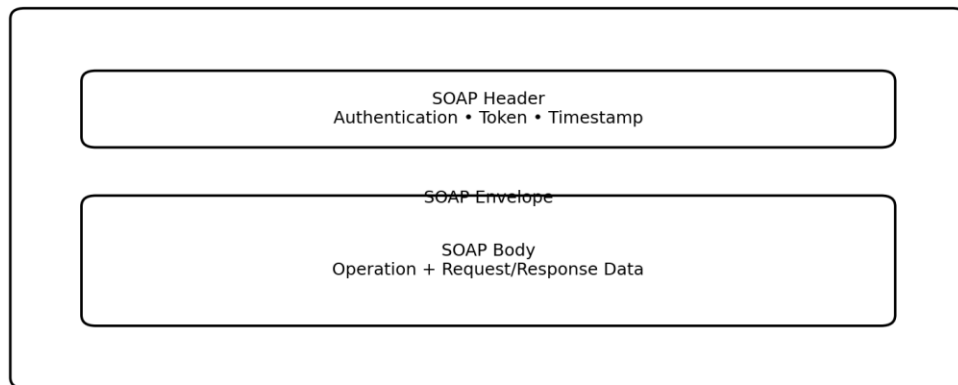


Figure 2. Envelope, Header and Body of a SOAP message.

Envelope: The root element that identifies the XML document as a SOAP message and encloses the Header and Body.

Header: Carries optional control information such as authentication credentials, security tokens, message IDs and timestamps.

Body: Contains the actual operation and business data, such as patient registration details or a medical-record request.

Example SOAP Request – Add Patient

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hos="http://hospital.example.com/">
  <soap:Header>
    <hos:Authentication>
      <hos:Username>reception01</hos:Username>
      <hos:Token>ABCD123456</hos:Token>
    </hos:Authentication>
  </soap:Header>
  <soap:Body>
  
```

```
<hos:AddPatient>
  <hos:PatientName>Arun Kumar</hos:PatientName>
  <hos:Age>35</hos:Age>
  <hos:Gender>Male</hos:Gender>
  <hos:BloodGroup>O+</hos:BloodGroup>
</hos:AddPatient>
</soap:Body>
</soap:Envelope>
```

Example SOAP Response

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hos="http://hospital.example.com/">
  <soap:Body>
    <hos:AddPatientResponse>
      <hos:PatientID>P10045</hos:PatientID>
      <hos:Status>Patient Registered Successfully</hos:Status>
    </hos:AddPatientResponse>
  </soap:Body>
</soap:Envelope>
```

Client–Server Data Exchange

1. Client creates a SOAP XML request according to the WSDL contract.
2. The request is sent to the service endpoint, normally over HTTPS.
3. The server parses the Envelope and validates security information in the Header.
4. The operation in the Body is identified and the corresponding business logic executes.
5. The service reads or updates the hospital database.
6. The server creates a SOAP response and returns it to the client.
7. If processing fails, a SOAP Fault provides standardized error information.

Justification for SOAP

SOAP is appropriate for this scenario because hospital data exchange requires strict structure and dependable processing. XML Schema can validate message formats; WSDL provides a precise service contract; HTTPS and WS-Security can provide confidentiality, authentication, integrity and digital signatures; SOAP Fault provides standardized errors; and enterprise SOAP standards can support reliable messaging and transaction-oriented workflows. These features make SOAP well suited to heterogeneous hospital systems where security, auditability and interoperability are important.

Question 2: WSDL Design for an Enterprise Web Service

WSDL (Web Services Description Language) is an XML-based contract that describes what a web service offers, the structure of its input and output data, how operations are invoked, the communication protocol used, and the endpoint where the service is available.

WSDL Service Contract Structure

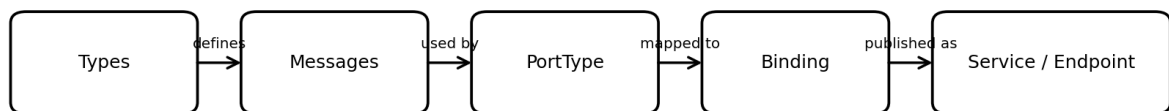


Figure 3. Relationship among the principal WSDL components.

Component	Role	Example
types	Defines request/response data structures, usually with XML Schema (XSD).	GetDataRequest / GetDataResponse
message	Defines logical input and output messages and their parts.	GetDataInput / GetDataOutput
portType	Defines the abstract service interface and its operations, such as GetData and UpdateData.	GetData(), UpdateData()

binding	Specifies how the abstract operations are transmitted, for example SOAP document/literal over HTTP.	SOAP over HTTP/HTTPS
service	Publishes one or more ports and gives the concrete endpoint address.	https://example.com/EnterpriseDataService

Illustrative WSDL Structure

```

<definitions name="EnterpriseDataService" ...>
  <types>
    <!-- XSD definitions for GetDataRequest, GetDataResponse,
         UpdateDataRequest and UpdateDataResponse -->
  </types>

  <message name="GetDataInput">...</message>
  <message name="GetDataOutput">...</message>

  <portType name="EnterpriseDataPortType">
    <operation name="GetData">...</operation>
    <operation name="UpdateData">...</operation>
  </portType>

  <binding name="EnterpriseDataBinding"
    type="tns:EnterpriseDataPortType">
    <soap:binding style="document"
      transport="http://schemas.xmlsoap.org/soap/http"/>
  </binding>

  <service name="EnterpriseDataService">

```



```
<port name="EnterpriseDataPort"
      binding="tns:EnterpriseDataBinding">
  <soap:address
    location="https://example.com/EnterpriseDataService"/>
</port>
</service>
</definitions>
```

How WSDL Enables Integration

The types and message sections establish exactly what data is exchanged. portType defines the operations without committing to a transport. binding maps those operations to SOAP and defines encoding/style details. service then publishes the concrete network address. External systems can therefore generate compatible client stubs, validate requests, discover required parameters and invoke the correct endpoint with much less ambiguity.

Importance

- Acts as a formal contract between service provider and consumer.
- Supports interoperability across programming languages and platforms.
- Enables automatic client/proxy generation in many development tools.
- Reduces integration errors by defining operations, messages, types and endpoints explicitly.
- Improves maintainability because interface changes can be managed through a versioned contract.

Question 3: Analysis and Debugging of SOAP Communication

When SOAP requests are rejected, delayed or produce incorrect responses, debugging should follow the complete communication path: message creation, transport, XML parsing, namespace resolution, operation dispatch, business logic, database access and response generation.

SOAP Debugging Workflow

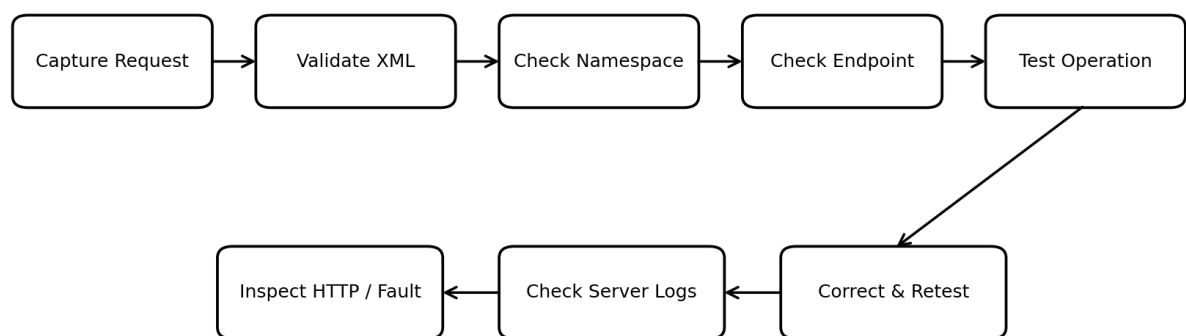


Figure 4. Systematic SOAP debugging workflow.

Problem	Effect	Debugging / Correction
Malformed XML	Missing closing tags, invalid nesting or illegal characters.	Validate the SOAP XML with an XML parser and schema.
Wrong namespace	The operation namespace differs from the WSDL target namespace.	Compare namespace URIs exactly with the WSDL.
Incorrect endpoint	Client sends to a wrong URL, path, protocol or port.	Verify soap:address, DNS, HTTPS configuration and firewall.

Wrong SOAPAction	Server dispatches to a different operation or rejects the request.	Match the action with the WSDL/binding configuration.
Data-type mismatch	A value violates the XSD type or required element order.	Validate request against the service XSD.
Authentication failure	Token, certificate, username or timestamp is invalid/expired.	Inspect security headers and server authentication logs.
Server/database delay	Slow queries, overload, deadlocks or downstream dependencies delay response.	Measure timings, inspect logs and optimize the slow component.

SOAP Fault Handling

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid Patient ID</faultstring>
  <detail>Patient ID must follow the required schema.</detail>
</soap:Fault>
```

A SOAP Fault should return a meaningful fault code and description without exposing sensitive internal details. Application logs can contain a correlation/message ID so administrators can trace the detailed cause securely.

Recommended Debugging Procedure

1. Read the current WSDL and confirm the expected operation, endpoint, SOAP version, namespaces and data types.
2. Capture the exact outbound SOAP request and inbound response.
3. Check that the XML is well-formed, then validate the Body content against the XSD.
4. Compare namespace URIs and operation names character-for-character with the WSDL.
5. Verify the endpoint, HTTP method, Content-Type and SOAPAction requirements.
6. Inspect HTTP status codes and SOAP Fault information.

7. Check authentication headers, certificates/tokens and timestamps.
8. Review server logs, database logs and processing time for delays or exceptions.
9. Correct one issue at a time and retest each operation independently.
10. Add automated schema validation, structured logging, timeouts and controlled retry policies to prevent recurrence.

How the Corrected System Ensures Reliability

After correction, each request conforms to the WSDL and XSD, reaches the correct endpoint, carries valid security information and maps to the intended operation. Standard SOAP Faults provide predictable error responses, while logging and correlation IDs make failures traceable. HTTPS protects transport data, schema validation prevents malformed inputs, and suitable timeout/retry mechanisms handle temporary communication failures. Together these controls improve correctness, security, observability and dependable communication.

Overall Conclusion

SOAP provides a standardized XML messaging framework suitable for enterprise and hospital integration where strict contracts, security and reliable error handling are important. WSDL describes the interface in a machine-readable form, while systematic validation and debugging ensure that SOAP clients and servers exchange accurate and dependable messages.